Ehime Tour Planner アプリ紹介レポート

Tavily と Gemini を用いた愛媛観光 RAG プランナー

matsuo_iwasawa

2026 年 4 月 20 日

(1) アプリの概要

本レポートでは，愛媛県の観光情報をもとに旅行プランを自動生成するアプリ `Ehime Tour Planner` を紹介する。参考資料である `example3_ChatGPT(LangChain).pdf` では，AI アプリを段階的に説明しながらコードを本文に埋め込む構成が採られていた。そこで本稿でも同様に，「入力条件の取得」「関連ページの検索」「RAG による関連情報の抽出」「構造化出力による旅程生成」「チャットによる再計画」という順に実装を紹介する。

このアプリの目的は，愛媛旅行を計画したい利用者が，旅行日数，同行者，移動手段，関心テーマ，訪問エリアなどを入力するだけで，出典 URL つきの現実的な旅程案を得られるようにすることである。情報源は主に `iyokannet.jp` を対象とし，原文を長く転載せず，要約とパラフレーズを通じて旅行プランに再構成する点が特徴である。

使用技術の要点

- UI: `Streamlit`
- 検索: `Tavily Search`（必要に応じて Web 全体も追加）
- 埋め込み: `gemini-embedding-001`
- 生成: `gemini-2.5-flash-lite`
- 検索基盤: `FAISS`，利用できない場合は NumPy のコサイン類似度で代替
- 出力: JSON スキーマ準拠の旅程と Markdown ダウンロード

(2) 方法

1. 入力条件を受け取る画面

まず，利用者が旅行条件を指定するための画面を `Streamlit` で構成する。旅行日数，開始日，同行者，移動手段，発着地，関心テーマ，訪問エリア，子連れ配慮，1 日の詰め込み度をサイドバーか

ら入力できるようにしている。この段階で条件を明示的に取得しておくことで，後段の検索クエリと旅程生成プロンプトに同じ制約を反映できる。

コード 1: 条件入力 UI（app.py より）

```
# --- Sidebar: 条件入力 ---
st.sidebar.header("プラン条件")
with st.sidebar:
    trip_days = st.number_input("旅行日数（日）", 1, 14, 2)
    start_date = st.date_input("開始日（任意）", value=date.today())
    party = st.text_input("同行者（例: 大人 2・小学生 1）", "大人 2")
    transport = st.selectbox("移動手段", ["公共交通", "自家用車", "レンタカー", "自転車"], index=1)

    st.divider()
    st.markdown("##### 発着地（任意）")
    start_end_options = ["指定なし", "松山空港", "JR 松山駅", "松山市駅", "松山観光港", "その他（自由記述）"]
    start_end_choice = st.selectbox("場所を選択", start_end_options, index=0)

    start_end_point = ""
    if start_end_choice == "その他（自由記述）":
        start_end_point = st.text_input("自由記述欄", placeholder="例: 今治港、自宅など")
    else:
        start_end_point = start_end_choice
    st.divider()

    interests = st.multiselect(
        "関心テーマ",
        ["温泉", "城・歴史", "サイクリング", "自然景観", "島めぐり", "グルメ", "アート", "祭り・イベント", "体験・アクティビティ"],
        default=["温泉", "グルメ"],
    )

    area_options = ["指定なし", "中予（松山・道後）", "東予（今治・西条など）", "南予（大洲・内子・宇和島など）", "その他（自由記述）"]
    area_choice = st.selectbox("主な訪問エリア", area_options, index=0)

    start_area = ""
    if area_choice == "その他（自由記述）":
        start_area = st.text_input("エリアを自由に入力", placeholder="例: 愛南町、鬼北町")
    else:
        start_area = area_choice

    with_kids = st.checkbox("子連れ考慮")
    pace = st.select_slider("1 日の詰め込み度", options=["ゆったり", "標準", "ぎっしり"], value="標準")
    generate_btn = st.button("プラン生成", type="primary")
```

2. 関連ページの収集と前処理

次に，旅行計画に必要な観光情報を集める。このアプリでは Tavily を用い，既定では iyokannet.jp にドメインを限定して検索する。必要に応じて Web 全体の検索結果も加えられるようにしつつ，URL の重複は除去し，取得した本文は BeautifulSoup でテキスト化してから保持する。これにより，出典を保ちつつも，そのままの転載ではない再利用可能な検索用データを作成できる。

コード 2: 検索と前処理（rag/retriever.py より）

```
    def search_and_prepare(self, query: str, max_results: int = 8, add_web_search: bool = False) -> List[RetrievalItem]:
        items: List[RetrievalItem] = []
        seen_urls = set()

        # 配分を決定
        if add_web_search:
            iyokan_results_count = max_results // 2
            web_results_count = max_results - iyokan_results_count
        else:
            iyokan_results_count = max_results
```

```
            web_results_count = 0

        def _process_results(results, is_iyokan: bool):
            for r in results:
                url = r.get("url", "")
                if not url or url in seen_urls:
                    continue

                title = r.get("title", "") or url
                raw_md = r.get("raw_content", "") or "\n".join(r.get("content", []))

                site_name = "いよ観ネット" if is_iyokan else url.split('/')[2].replace("www.", "")

                cleaned = self._clean_text(raw_md)
                if not cleaned:
                    try:
                        ext = self.client.extract(url)
                        cleaned = self._clean_text(ext.get("text", ""))
                    except Exception:
                        continue

                if cleaned:
                    items.append(RetrievalItem(
                        title=title[:180],
                        url=url,
                        site=site_name,
                        content=cleaned[:10000],
                        content_chars=len(cleaned),
                    ))
                    seen_urls.add(url)

        # 1. いよ観ネットを検索
        if iyokan_results_count > 0:
            try:
                resp_iyokan = self.client.search(
                    query=query, search_depth="advanced", include_raw_content="markdown",
                    include_answer=False, include_domains=["iyokannet.jp"],
                    max_results=iyokan_results_count, chunks_per_source=3, timeout=120
                )
                _process_results(resp_iyokan.get("results", []), is_iyokan=True)
            except Exception as e:
                print(f"Error searching iyokannet.jp: {e}")

        # 2. ウェブ全体を検索（追加が有効な場合）
        if web_results_count > 0:
            try:
                resp_web = self.client.search(
                    query=query, search_depth="advanced", include_raw_content="markdown",
                    include_answer=False, max_results=web_results_count,
                    chunks_per_source=3, timeout=120
                )
                _process_results(resp_web.get("results", []), is_iyokan=False)
            except Exception as e:
                print(f"Error searching web: {e}")

        return items

    def _clean_text(self, text: str) -> str:
        if not text:
            return ""
        # Markdown/HTML → テキスト
        soup = BeautifulSoup(text, "html.parser")
        txt = soup.get_text(separator="n")
        txt = html.unescape(txt)
        txt = re.sub(r"n{3,}", "nn", txt)
        txt = re.sub(r"s+", " ", txt)
        # いよ観ネットの原文転載を避けるため、チャンク化前に短縮
        return txt.strip()
```

3. 埋め込みとベクトル検索による RAG

収集したテキストはそのまま生成器に渡さず，一度チャンクに分割して埋め込みベクトルへ変換する。本アプリでは gemini-embedding-001 を使い，FAISS が利用可能であればそのインデックスを用い，利用できない環境では NumPy によるコサイン類似度計算へ切り替える。この実装により，環境差に強い RAG パイプラインを保ちながら，ユーザーの検索語に近い文脈だけを抽出できる。

コード 3: 埋め込みと検索基盤（rag/retriever.py より）

```
    # --- 2) 埋め込みユーティリティ ---
    def _embed(self, texts: List[str], task_type: str, dim: int = 768) -> np.ndarray:
        all_vecs = []
        # Process in batches to respect API limits (e.g., 100 texts per batch)
        # and add a delay to respect rate limits (e.g., 60 requests per minute)
        batch_size = 100

        for i in range(0, len(texts), batch_size):
            batch_texts = texts[i : i + batch_size]
            if not batch_texts:
                continue

            cfg = types.EmbedContentConfig(task_type=task_type, output_dimensionality=dim)
            try:
                res = self.gclient.models.embed_content(
                    model="gemini-embedding-001",
                    contents=batch_texts,
                    config=cfg,
                )

                batch_vecs = [np.array(e.values, dtype="float32") for e in res.embeddings]
                all_vecs.extend(batch_vecs)

                # If there are more batches to process, wait to avoid hitting rate limits.
                if i + batch_size < len(texts):
                    print(f"Embedding batch {i//batch_size + 1} complete. Waiting for 5 seconds...")
                    time.sleep(5)

            except Exception as e:
                print(f"An error occurred during embedding batch {i//batch_size + 1}: {e}")
                # Continue to the next batch if one fails
                continue

        if not all_vecs:
            # Handle case where all embedding calls failed
            return np.array([], dtype="float32").reshape(0, dim)

        return np.vstack(all_vecs)

    # --- 3) ベクトル化 → 検索 ---
    def _build_index(self, chunks: List[str]):
        X = self._embed(chunks, task_type="RETRIEVAL_DOCUMENT")
        if USE_FAISS:
            index = faiss.IndexFlatIP(X.shape[1])
            faiss.normalize_L2(X)
            index.add(X)
            return index, X
        # 代替: 生行列を返す
        return None, X

    def _search_index(self, index, X: np.ndarray, q: np.ndarray, topk: int = 8) -> Tuple[List[int], List[float]]:
        if USE_FAISS and index is not None:
            faiss.normalize_L2(q)
            D, I = index.search(q, topk)
            return I[0].tolist(), D[0].tolist()
        # 代替: NumPy コサイン類似度
        # 正規化
        Xn = X / (np.linalg.norm(X, axis=1, keepdims=True) + 1e-9)
        qn = q / (np.linalg.norm(q, axis=1, keepdims=True) + 1e-9)
        sims = Xn @ qn.T
        order = np.argsort(-sims[:, 0])[:topk]
```

```
        return order.tolist(), sims[order, 0].tolist()
```

さらに retrieve_for_plan では，単に類似度の高いチャンクを並べるのではなく，「1 URL あたり 1 チャンク」に絞ることで情報源の偏りを抑えている。また，選ばれたチャンク群は Gemini に一括で要約させる。このまとめ要約は API 呼び出し回数の削減につながるだけでなく，観光記事の原文を長く引用しないという方針にも合致している。

コード 4: チャンク選択と一括要約（rag/retriever.py より）

```
    def retrieve_for_plan(self, items: List[RetrievalItem], user_query: str, k: int = 8):
        chunk_texts, chunk_meta = [], []
        for it in items:
            chunks = self._chunk(it.content)
            for ch in chunks:
                chunk_texts.append(ch)
                chunk_meta.append({"title": it.title, "url": it.url, "site": it.site})

        if not chunk_texts:
            return [], []

        index, X = self._build_index(chunk_texts)
        if X.shape[0] == 0:
            return [], []

        q = self._embed([user_query], task_type="RETRIEVAL_QUERY")
        ids, scores = self._search_index(index, X, q, topk=k)

        # 1）まず「URL あたり 1 チャンク」に絞る（ここが効く）
        picked = []
        used_sources = []
        seen_urls = set()

        for idx in ids:
            meta = chunk_meta[idx]
            url = meta["url"]
            if url in seen_urls:
                continue
            seen_urls.add(url)
            used_sources.append(meta)
            picked.append(idx)
            # 無料枠前提なら 4〜5 程度が安全（任意で調整）
            if len(picked) >= min(k, 5):
                break

        if not picked:
            return [], []

        # 2）要約をまとめて 1 回だけ実行
        texts_to_sum = [chunk_texts[idx] for idx in picked]
        print(f"Summarizing {len(texts_to_sum)} chunks in one request...")
        summaries = self._summarize_for_context_batch(texts_to_sum)

        selected = []
        for idx, summary in zip(picked, summaries):
            meta = chunk_meta[idx]
            selected.append(f"出典: {meta['title']} | {meta['url']}\n 要点:\n{summary}")

        return selected, used_sources


    def _summarize_for_context(self, text: str) -> str:
        # いよ観ネットのポリシーに配慮: 引用ではなく短い要点箇条書き
        prompt = (
            "以下の観光記事テキストから、固有名詞と実用情報（場所・体験・時期・所要時間・注意点）を日本語で 5 点以内に簡潔要約してください。n"
            "**原文の連続した引用は禁止。必ず言い換え・要約で**。最大 400 字。nn" + text[:4000]
        )
        resp = self.gclient.models.generate_content(
            model="gemini-2.5-flash-lite",
            contents=prompt,
            config=types.GenerateContentConfig(
                thinking_config=types.ThinkingConfig(thinking_budget=0),
            ),
```

```
        )
        return resp.text.strip()

    def _summarize_for_context_batch(self, texts: List[str]) -> List[str]:
        """
        複数チャンクを 1 回の generate_content で要約して返す（順序維持）。
        返り値は texts と同じ長さの summary リスト。
        """
        # JSON で返させる（順序通りの配列）
        joined = "\n\n".join(
            [f"### CHUNK {i+1}\n{t[:4000]}" for i, t in enumerate(texts)]
        )

        prompt = (
            "以下の複数の観光記事テキストを、それぞれ日本語で要点要約してください。\n"
            "- 各チャンクは固有名詞と実用情報（場所・体験・時期・所要時間・注意点）を中心に 5 点以内\n"
            "- 原文の連続した引用は禁止。必ず言い換え・要約で\n"
            "- 1 チャンクあたり最大 400 字\n"
            "出力は必ず JSON で、次の形式にしてください：\n"
            '{"summaries": ["要約 1", "要約 2", "..."]}\n\n'
            + joined
        )

        # 503/429 対策：軽いリトライ（必要最小限）
        last_err = None
        for attempt in range(4):
            try:
                resp = self.gclient.models.generate_content(
                    model="gemini-2.5-flash-lite",
                    contents=prompt,
                    config={
                        "response_mime_type": "application/json",
                        # thinking を切って軽量化したい場合（任意）
                        "thinking_config": {"thinking_budget": 0},
                    },
                )
                data = json.loads(resp.text)
                summaries = data.get("summaries", [])
                # 長さ不一致の保険
                if not isinstance(summaries, list) or len(summaries) != len(texts):
                    return ["" for _ in texts]
                return [str(s).strip() for s in summaries]
            except Exception as e:
                last_err = e
                msg = str(e)
                # 429/503 の場合だけ待って再試行
                if ("429" in msg) or ("RESOURCE_EXHAUSTED" in msg) or ("503" in msg) or ("UNAVAILABLE" in msg) or ("overloaded" i
                    time.sleep(min(60, 2 ** attempt * 4))
                    continue
                raise
        raise last_err
```

4. プロンプト設計と構造化出力

旅程生成では，自由文だけを返させるのではなく，JSON スキーマに従った構造化出力を要求している。プロンプト側では，長い原文引用の禁止，各日ごとの根拠 URL の提示，安全面や季節性への配慮などをガードレールとして明示する。また，発着地や宿泊地の扱いを具体的に指定することで，単なる観光地の列挙ではなく，日程として実行しやすいプランになるよう工夫している。

コード 5: 旅程生成プロンプト（rag/prompts.py より）

```
SYSTEM_GUARDRAILS = (
    "あなたは愛媛旅行の日本語プランナー。いよ観ネットの要約から**パラフレーズ**で情報を統合する。\n"
    "禁止: 原文の長い引用・転載、無根拠の情報。\n"
    "必須: 各日ごとに根拠 URL を列挙し、家族/高齢者/子連れなど安全面と季節性を明示。\n"
)

TEMPLATE = '''{system}
```

```
【旅行条件】
- 日数: {trip_days}日
- 開始日: {start_date}
- 同行者: {party}
- 移動手段: {transport}
- 関心テーマ: {interests}
- 主な訪問エリア: {start_area}
- 子連れ配慮: {with_kids}
- ペース: {pace}
- 発着地: {start_end_point}

【参考要点】
{context}

【指示】
上記条件に基づき、以下の点を厳守して現実的な旅行プランを JSON 形式で作成してください。

1.  **行程の起点と終点:** 1 日目は「{start_end_point}」から出発し、最終日（{trip_days}日目）は「{start_end_point}」に到着して解散する行程とし
2.  **宿泊地の最適化:** **毎日、発着地に戻る必要はありません。** 各日の宿泊地は、その日の観光エリアや翌日の移動を考慮して、最も効率的で現実的な場
3.  **時間配分:** 各アクティビティの所要時間と、エリア間の移動時間を考慮した、現実的な時間割を作成してください。
4.  **出力形式:** 必ず指定された JSON スキーマに従ってください。
'''

def build_plan_prompt(
    trip_days: int,
    start_date: str,
    party: str,
    transport: str,
    interests: list[str],
    start_area: str,
    with_kids: bool,
    pace: str,
    start_end_point: str,
    sources: list[dict],
    context: list[str],
) -> str:
    ctx = "\n\n".join(context)
    system = SYSTEM_GUARDRAILS

    start_end_prompt_val = start_end_point if start_end_point and start_end_point != "指定なし" else "指定なし"

    return TEMPLATE.format(
        system=system,
        trip_days=trip_days,
        start_date=start_date,
        party=party,
        transport=transport,
        interests=",".join(interests),
        start_area=start_area,
        with_kids=with_kids,
        pace=pace,
        start_end_point=start_end_prompt_val,
        context=ctx,
    )
```

実際の生成処理では，上記プロンプトと抽出済みコンテキストを generate_content に渡し，response_json_schema を指定して旅程の構造を固定する。これにより，生成結果をそのまま画面表示や Markdown 出力へつなげやすくなる。

コード 6: 初回プラン生成（app.py より）

```
    with st.spinner("RAG で関連チャンクを選別中..."):
        top_chunks, used_sources = retriever.retrieve_for_plan(
            items=[RetrievalItem(**i) for i in items_state],
            user_query=query,
            k=8,
        )

    with st.spinner("Gemini で旅程を構成中..."):
        prompt = build_plan_prompt(
            trip_days=trip_days, start_date=str(start_date), party=party,
            transport=transport, interests=interests, start_area=start_area,
            with_kids=with_kids, pace=pace, start_end_point=start_end_point,
```

```
            sources=used_sources, context=top_chunks,
        )
        resp = client.models.generate_content(
            model="gemini-2.5-flash-lite",
            contents=prompt,
            config={
            "response_mime_type": "application/json",
            "response_json_schema": ITINERARY_SCHEMA,  # JSON Schema (dict)
            },
        )

        st.session_state.plan_json = json.loads(resp.text)
        st.session_state.messages = [{
            "role": "assistant",
            "content": "プランの初稿を作成しました。変更したい点があれば、下のチャット欄から具体的に教えてください。\n（例: 2日目はもっとゆった
        }]
        st.rerun()
```

5. チャットによる再計画

参考 PDF では，単一の処理だけでなく，複数段階の処理やエージェント化が順を追って説明されていた。本アプリでも同じ考え方で，初回生成のあとにチャットで修正依頼を受け取り，既存プランを保ちながら再構成する流れを用意している。利用者は「2 日目をもっとゆったりにしたい」「温泉を追加したい」といった自然文で修正を依頼できる。

コード 7: チャットによるプラン修正（app.py より）

```
if prompt := st.chat_input("プランの修正点を入力してください"):
    if not st.session_state.plan_json:
        st.warning("まずプランを生成してください。")
        st.stop()

    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    with st.chat_message("assistant"):
        with st.spinner("プランを修正中..."):
            refine_prompt = build_refine_plan_prompt(
                existing_plan=st.session_state.plan_json,
                user_request=prompt,
            )
            resp = client.models.generate_content(
                model="gemini-2.5-flash-lite",
                contents=refine_prompt,
                config={
                    "response_mime_type": "application/json",
                    "response_json_schema": ITINERARY_SCHEMA,
                },
            )

            try:
                new_plan = json.loads(resp.text)
                st.session_state.plan_json = new_plan
                response_text = "プランを修正しました。いかがでしょうか？ さらに修正したい点があれば、教えてください。"
            except json.JSONDecodeError:
                response_text = "プランの修正に失敗しました。形式が正しくないようです。もう一度試しますか？\n" + resp.text

            st.session_state.messages.append({"role": "assistant", "content": response_text})
            st.rerun()
```

(3) わかったこと・今後の改善

このアプリを紹介する中で，次の点が重要だと分かった。

- 検索対象を `iyokannet.jp` 中心に絞ることで，観光プランの根拠を明確にしやすい。
- 生成前に関連チャンクを選別し，要点だけをまとめて渡すことで，長い観光記事をそのまま扱うよりも安定した旅程が得られる。
- JSON スキーマによる構造化出力は，表示，保存，再編集のすべてに有効であり，単なるテキスト生成より実用性が高い。
- チャットで再計画できる構成にしておくと，初回出力をたたき台として徐々に利用者の意図へ寄せられる。

一方で，今後の改善点もある。第一に，移動時間は現在のプロンプトと記事要約に大きく依存しているため，地図 API や経路探索を組み合わせれば，さらに現実的な時間割に近づけられる。第二に，検索対象が観光記事中心であるため，休館日や季節イベントの変化に対しては追加の最新確認が必要である。第三に，宿泊候補や費用見積もりを明示的に扱うと，より旅行計画アプリらしい完成度になると考えられる。

参考資料

- `matsuo_iwasawa/example3_ChatGPT(LangChain).pdf`
- `README.md`
- `app.py`
- `rag/retriever.py`
- `rag/prompts.py`
- `utils/formatting.py`